University of Science, VNU-HCM

CSC14003 - Introduction to Artificial Intelligence

Project 2 - Futoshiki Logic Solver

Course:	CSC14003 - Introduction to Artificial Intelligence
Project:	Project 2 - Futoshiki Logic Solver
Entry point:	`main.py`
GUI framework:	Streamlit web application
Date:	April 15, 2026

Team Information
21127645 - Le Minh - `lminh21@clc.fitus.edu.vn`
20127119 - Pham Nguyen Gia Bao - `pngbao20@clc.fitus.edu.vn`
19127440 - Tran Hoang Ngan Khanh - `thnkhanh19@clc.fitus.edu.vn`
21127713 - Nguyen Son Truong - `nstruong21@clc.fitus.edu.vn`

Contents

1 Introduction to Futoshiki

Futoshiki is a logic puzzle played on an $N \times N$ grid. Each row and each column must contain every number from 1 to N exactly once, and adjacent cells connected by inequality signs must satisfy those signs. Some cells are given at the start and must stay unchanged. Compared with Sudoku, Futoshiki is smaller in structure but richer in explicit ordering constraints, so it is a good fit for first-order logic, forward chaining, backward chaining, and search-based solving.

This project implements the required solvers for the CSC14003 logic assignment: ground knowledge-base generation, CNF conversion, forward chaining, backward chaining with SLD resolution, A* search, brute-force search, backtracking with CSP heuristics, benchmark scripts, charts, and a simple GUI for demonstration.

2 Project Planning and Task Distribution

2.1 Work Packages

The work was divided across four members with equal weight. Each member carried one main implementation block together with one verification or documentation block so that the contribution stayed balanced.

Member	Share	Assigned work
Le Minh	25%	Input parser, output formatter, validator core logic, brute-force solver, bundled input-output consistency checking, and unit tests for parsing, formatting, validation, and the brute-force baseline.
Pham Nguyen Gia Bao	25%	CSP backtracking solver with MRV, degree heuristic, forward checking, AC-3, and solver-side correctness testing for the main complete search solver.
Tran Hoang Ngan Khanh	25%	FOL formalization, ground knowledge-base generation, CNF conversion, forward chaining solver, backward chaining solver with SLD resolution, and the logic sections of the report.
Nguyen Son Truong	25%	A* solver, heuristic analysis, benchmark automation, chart generation, GUI integration, root entry point, README cleanup, and final packaging.

2.2 Detailed Task Table

Task group	Le Minh	Pham Nguyen Gia Bao	Tran Hoang Ngan Khanh	Nguyen Son Truong
Core coding	Parser, formatter, validator, brute-force solver	Backtracking solver implementation	Forward chaining and backward chaining solvers, CNF pipeline	A* solver, GUI flow, benchmark/chart pipeline
Verification	Parser, validator, and brute-force tests; bundled output checking	Backtracking and CSP propagation tests; solver-output cross-checks	Logic and CNF tests; contradiction handling checks	Benchmark reruns, chart checks, A* comparison checks, UI smoke checks
Documentation	Input-output format writeup and brute-force baseline notes	Search/CSP algorithm description and backtracking pseudocode review	FOL axioms, CNF derivations, forward/backward inference pseudocode	Heuristic analysis, experiment section, packaging notes
Integration	CLI consistency, input-output path handling	Backtracking output integration with validator and outputs	Logic solver integration with report claims	Root `main.py`, README, figures, final runbook

3 Problem Formulation

3.1 Puzzle Definition

An $N \times N$ Futoshiki puzzle contains:

- a grid of cells (i, j), with $1 \leq i, j \leq N$;
- given clues, where some cells are pre-filled with a value in $\{1, \ldots, N\}$;
- horizontal inequality symbols between (i, j) and $(i, j + 1)$;
- vertical inequality symbols between (i, j) and $(i + 1, j)$.

A valid solution must satisfy all of the following:

1. each row is a permutation of $1, \ldots, N$;
2. each column is a permutation of $1, \ldots, N$;
3. each horizontal and vertical inequality is true;
4. each given clue is preserved.

3.2 Search Formulation

Following the course material on problem solving by search, the A* formulation used in this repository is:

- **State**: a partial assignment represented as a domain map after propagation, not just a raw grid.
- **Initial state**: the puzzle with given clues and initial domains $\{1, \ldots, N\}$ for empty cells, then reduced by AC-3.
- **Action**: choose one ambiguous cell and assign one legal value from its current domain.
- **Result**: the new domain state after the assignment, forward checking, and AC-3.
- **Goal test**: every cell has a singleton domain and the resulting grid passes validation.
- **Path cost**: the number of branching decisions made so far.

3.3 CSP Formulation

Futoshiki is also a CSP:

- **Variables**: one variable per cell.
- **Domains**: $\{1, \ldots, N\}$ initially, or a pruned subset after propagation.
- **Constraints**: row all-different, column all-different, given clues, and horizontal/vertical inequalities.
- **Constraint graph**: nodes are cells, edges connect row peers, column peers, and inequality neighbors.

4 FOL Formalization

4.1 Vocabulary

The report follows the project AI - Project 2.pdf vocabulary:

$Val(i, j, v)$	: cell (i, j) has value v
$Given(i, j, v)$	: the puzzle gives value v at (i, j)
$LessH(i, j)$	: $(i, j) < (i, j + 1)$
$GreaterH(i, j)$	: $(i, j) > (i, j + 1)$
$LessV(i, j)$	: $(i, j) < (i + 1, j)$
$GreaterV(i, j)$	: $(i, j) > (i + 1, j)$
$Less(v_1, v_2)$	: arithmetic background fact $v_1 < v_2$

The implementation also uses auxiliary predicates for finite-domain inference:

$$Possible(i, j, v), \quad NotVal(i, j, v), \quad Assigned(i, j).$$

4.2 Core Axioms

Let the index and value domain be $\{1, \ldots, N\}$.

A1. Every cell has at least one value.

$$\forall i \forall j \exists v \, Val(i, j, v)$$

A2. Every cell has at most one value.

$$\forall i \forall j \forall v_1 \forall v_2 \, \big(Val(i, j, v_1) \wedge Val(i, j, v_2)\big) \Rightarrow (v_1 = v_2)$$

A3. Row uniqueness.

$$\forall i \forall j_1 \forall j_2 \forall v \, \big(Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2\big) \Rightarrow \bot$$

A4. Column uniqueness.

$$\forall j \forall i_1 \forall i_2 \forall v \, \big(Val(i_1, j, v) \wedge Val(i_2, j, v) \wedge i_1 \neq i_2\big) \Rightarrow \bot$$

A5. Given clues are enforced.

$$\forall i \forall j \forall v \; Given(i, j, v) \Rightarrow Val(i, j, v)$$

A6. Horizontal less-than.

$$\forall i \forall j \forall v_1 \forall v_2 \, \big(LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j+1, v_2)\big) \Rightarrow Less(v_1, v_2)$$

A7. Horizontal greater-than.

$$\forall i \forall j \forall v_1 \forall v_2 \, \big(GreaterH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j+1, v_2)\big) \Rightarrow Less(v_2, v_1)$$

A8. Vertical less-than.

$$\forall i \forall j \forall v_1 \forall v_2 \, \big(LessV(i, j) \wedge Val(i, j, v_1) \wedge Val(i+1, j, v_2)\big) \Rightarrow Less(v_1, v_2)$$

A9. Vertical greater-than.

$$\forall i \forall j \forall v_1 \forall v_2 \, \big(GreaterV(i, j) \wedge Val(i, j, v_1) \wedge Val(i+1, j, v_2)\big) \Rightarrow Less(v_2, v_1)$$

A10. Domain restriction and background order. Only values in $\{1, \ldots, N\}$ are grounded. The implementation explicitly generates the background facts

$$Less(a, b) \quad \text{for all } 1 \leq a < b \leq N.$$

This finite grounding plays the role of the domain restriction/completeness axiom in the code.

5 CNF Derivation for Three Non-trivial Axioms

5.1 Axiom 1: Every Cell Has At Least One Value

Start from:

$$\forall i \forall j \exists v \, Val(i, j, v)$$

Step 1: Standardize variables.

$$\forall i_0 \forall j_1 \exists v_2 \, Val(i_0, j_1, v_2)$$

Step 2: Skolemization. The existential variable depends on the universally quantified variables i_0 and j_1, so it becomes a Skolem function, not a constant:

$$\forall i_0 \forall j_1 \, Val(i_0, j_1, Sk_0(i_0, j_1))$$

Step 3: Drop universal quantifiers.

$$Val(i_0, j_1, Sk_0(i_0, j_1))$$

Step 4: Finite grounding used in the implementation. Because the actual puzzle domain is finite and known, the code replaces the existential witness by explicit disjunction over $1, \ldots, N$:

$$Val(i, j, 1) \vee Val(i, j, 2) \vee \cdots \vee Val(i, j, N)$$

For a 4×4 cell, this becomes:

$$Val(i, j, 1) \vee Val(i, j, 2) \vee Val(i, j, 3) \vee Val(i, j, 4)$$

5.2 Axiom 3: Row Uniqueness

Start from:

$$\forall i \forall j_1 \forall j_2 \forall v \, \big(Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2\big) \Rightarrow \bot$$

Step 1: Remove implication.

$$\forall i \forall j_1 \forall j_2 \forall v \, \neg\big(Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2\big)$$

Step 2: Push negation inward.

$$\forall i \forall j_1 \forall j_2 \forall v \, \big(\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v) \vee j_1 = j_2\big)$$

Step 3: Ground only the meaningful case $j_1 \neq j_2$. For distinct columns, the equality literal is false and disappears. The grounded clause is:

$$\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v)$$

Example. For row 1, columns 1 and 3, value 2:

$$\neg Val(1, 1, 2) \vee \neg Val(1, 3, 2)$$

5.3 Axiom 6: Horizontal Less-than

Start from:

$$\forall i \forall j \forall v_1 \forall v_2 \ \big(LessH(i,j) \land Val(i,j,v_1) \land Val(i,j+1,v_2)\big) \Rightarrow Less(v_1,v_2)$$

Step 1: Remove implication.

$$\forall i \forall j \forall v_1 \forall v_2 \ \big(\neg LessH(i,j) \lor \neg Val(i,j,v_1) \lor \neg Val(i,j+1,v_2) \lor Less(v_1,v_2)\big)$$

Step 2: CNF form. This is already a single CNF clause.

Step 3: Ground and simplify using extensional order facts. Suppose the puzzle contains $LessH(1,1)$. For the invalid value pair $(v_1,v_2) = (4,2)$, the grounded clause is:

$$\neg LessH(1,1) \lor \neg Val(1,1,4) \lor \neg Val(1,2,2) \lor Less(4,2)$$

Since $LessH(1,1)$ is true and $Less(4,2)$ is false, the clause reduces to:

$$\neg Val(1,1,4) \lor \neg Val(1,2,2)$$

This is exactly the kind of forbidden-pair clause that appears in the direct propositional encoding.

6 Algorithms

6.1 Ground Knowledge Base and CNF Generation

The repository contains two related but separate representations:

1. a grounded Horn knowledge base for forward and backward chaining;
2. a direct propositional CNF encoding for verification.

This separation is deliberate. Horn inference is used for the required forward/backward chaining tasks, while CNF is used to verify final solutions against the propositionalized puzzle constraints.

6.2 Forward Chaining

The forward chaining engine is agenda-based and implemented from scratch over ground Horn rules. Facts are inserted into an agenda; whenever all premises of a rule are known, its head is derived and pushed back into the agenda.

Algorithm 1 Forward Chaining on a Ground Horn KB

```
facts ← initial facts
agenda ← queue of all initial facts
while agenda not empty do
    fact ← pop from agenda
    for each rule whose body contains fact do
        if all body atoms of the rule are in facts then
            derive the head atom
            if the head is new then
                add the head to facts and agenda
            end if
        end if
    end for
end while
return facts
```

In this project, the Horn KB derives Val, $NotVal$, $Possible$, and contradiction atoms from givens, row/column uniqueness, singleton domains, and inequality restrictions.

6.3 Backward Chaining with SLD Resolution

The backward chaining engine uses first-order terms, substitutions, unification, and depth-first SLD-style goal reduction. It is not a renamed CSP solver. The query engine really proves goals by matching facts and Horn-rule heads, then recursively proving the body.

Algorithm 2 Backward Chaining / SLD Resolution

```
function PROVE(goals, substitution)
    if goals is empty then
        return the current substitution
    end if
    g ← first goal after applying the current substitution
    for each fact or rule head that unifies with g do
        compute the most general unifier θ
        if unification succeeds then
            replace g by the rule body (or nothing for a fact)
            recurse on the new goal list under the extended substitution
        end if
    end for
end function
```

To keep the backward solver honest and still practical, the repository first runs propagation, then builds a compact Horn snapshot:

- $Given(i, j, v)$, $Possible(i, j, v)$, and $NotVal(i, j, v)$ are stored as facts;
- $Val(i, j, v)$ is proved by rules such as $Given(i, j, v) \Rightarrow Val(i, j, v)$ and the singleton-domain rule.

This matters academically: the current code no longer reads solved singleton values directly as `Val` facts during backward chaining.

6.4 Backtracking

The CSP solver is classical backtracking search with MRV, degree heuristic, forward checking, and AC-3. It is the cleanest complete reference solver in the repository.

Algorithm 3 Backtracking with Propagation

```
function SEARCH(domains)
    if all domains are singleton then
        return solution
    end if
    choose an unassigned cell by MRV, break ties by degree
    for each value in the chosen domain do
        assign the value
        run forward checking and AC-3
        if no contradiction then
            recurse
        end if
    end for
    return failure
end function
```

6.5 A* Search

The A* solver uses the search formulation from Section 3.

Algorithm 4 A* on Propagated Domain States

```
initialize the open set with the propagated start state
while open set not empty do
    pop the state with minimum f(n) = g(n) + h(n)
    if all domains are singleton then
        return solution
    end if
    pick one ambiguous cell
    for each legal value of that cell do
        assign the value, propagate, and compute g', h', f'
        push the new state if it improves the best known path cost
    end for
end while
return failure
```

6.6 Heuristic Design and Admissibility

Three heuristics were tested:

- $h_0(s) = 0$;
- $h_{weak}(s) = 1$ if any ambiguity remains, else 0;
- $h_{main}(s)$: the number of ambiguous connected components after propagation.

Why h_{main} is admissible. The proof follows the state definition used by the solver.

1. Before evaluating the heuristic, the state is reduced by AC-3 to a fixed point.
2. Each remaining ambiguous connected component contains at least one cell with a non-singleton domain.
3. One action assigns one cell.
4. Two disconnected components in the active binary-constraint graph cannot resolve each other through propagation.
5. Therefore each ambiguous component needs at least one future branching decision.

So $h_{main}(s)$ is a lower bound on the remaining path cost measured in branching decisions.

7 Experimental Setup

7.1 Dataset

The benchmark uses ten bundled inputs, satisfying the project requirement for sizes 4×4, 5×5, 6×6, 7×7, and 9×9.

Input	Size	Givens	Horizontal signs	Vertical signs
input-01.txt	4	8	12	12
input-02.txt	4	6	12	12
input-03.txt	5	10	20	20
input-04.txt	5	8	20	20
input-05.txt	6	14	30	30
input-06.txt	6	12	30	30
input-07.txt	7	18	42	42
input-08.txt	7	16	42	42
input-09.txt	9	28	72	72
input-10.txt	9	24	72	72

7.2 Execution

All benchmark data in this report was generated by:

```
python -m futoshiki.cli benchmark --inputs inputs --out reports/benchmark_results.csv
```

The benchmark records:

- runtime in milliseconds;
- nodes expanded;
- rule firings or backward-chaining goal resolutions;

- a memory proxy.

No operating-system memory sampler was used. The memory proxy is the maximum among:

- `peak_frontier` for search depth/frontier growth;
- `peak_open_set` for A*;
- `peak_domain_size_sum` for the propagated-domain representation.

8 Results

8.1 Summary Table

Solver	Avg runtime (ms)	Max runtime (ms)	Avg nodes expanded	Avg inference count	Avg memory proxy
bruteforce	0.205	0.449	27.0	0.0	28.0
backtracking	4.555	13.751	0.0	0.0	209.0
astar-h0	4.647	14.098	1.0	0.0	41.4
astar-main	4.947	16.053	1.0	0.0	41.4
logic-forward	321.834	985.941	0.0	1441.8	41.4
logic-backward	676.373	2623.101	0.0	1528.0	41.4

8.2 Runtime per Input

Input	brute	backtrack	A*-h0	A*-main	logic-FC	logic-BC
input-01	0.061	0.540	0.469	0.550	39.320	15.673
input-02	0.066	0.506	0.497	0.555	23.655	15.428
input-03	0.102	1.173	1.228	1.257	97.193	55.148
input-04	0.113	1.229	1.390	1.352	85.392	86.329
input-05	0.145	2.904	2.490	2.812	195.444	207.426
input-06	0.161	2.808	3.048	3.409	156.754	190.143
input-07	0.214	4.568	4.886	4.911	346.406	473.146
input-08	0.307	5.092	4.870	5.112	325.451	499.764
input-09	0.435	12.978	13.498	13.461	962.785	2597.574
input-10	0.449	13.751	14.098	16.053	985.941	2623.101

8.3 Nodes Expanded and Inference Counts

For the current ten-input dataset:

- brute-force expands between 8 and 57 nodes, depending on puzzle size;
- backtracking expands 0 search nodes after initial propagation because AC-3 already forces singleton domains on these inputs;

- both A* variants expand exactly 1 node per instance for the same reason;
- forward chaining fires between 314 and 3561 rules;
- backward chaining resolves between 216 and 4311 goals.

8.4 Charts

The repository now generates charts directly from the benchmark CSV into `reports/figures/`. In Overleaf, upload the `figures/` folder and keep the filenames unchanged.

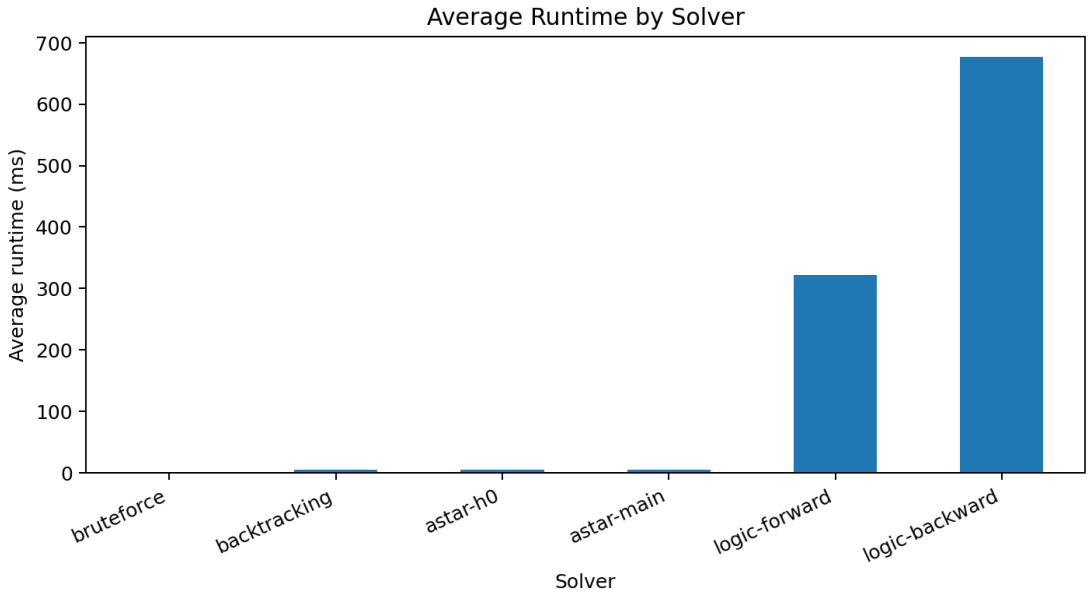

Figure 1: Average runtime by solver.

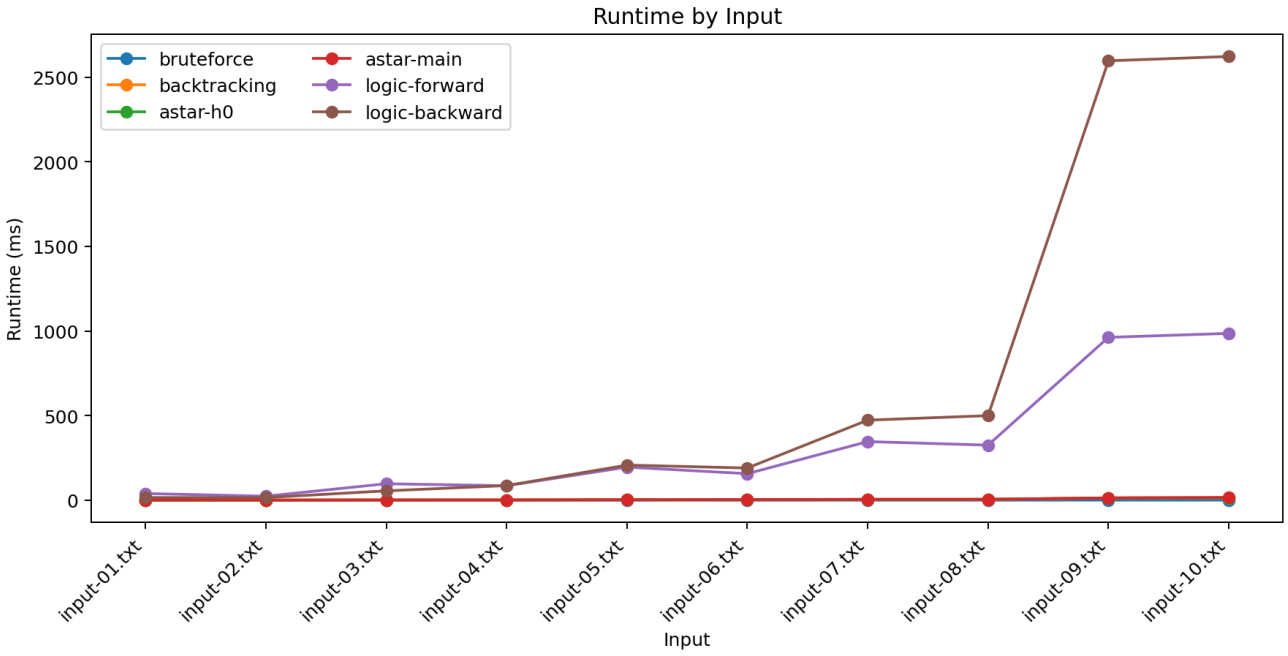

Figure 2: Runtime by input.

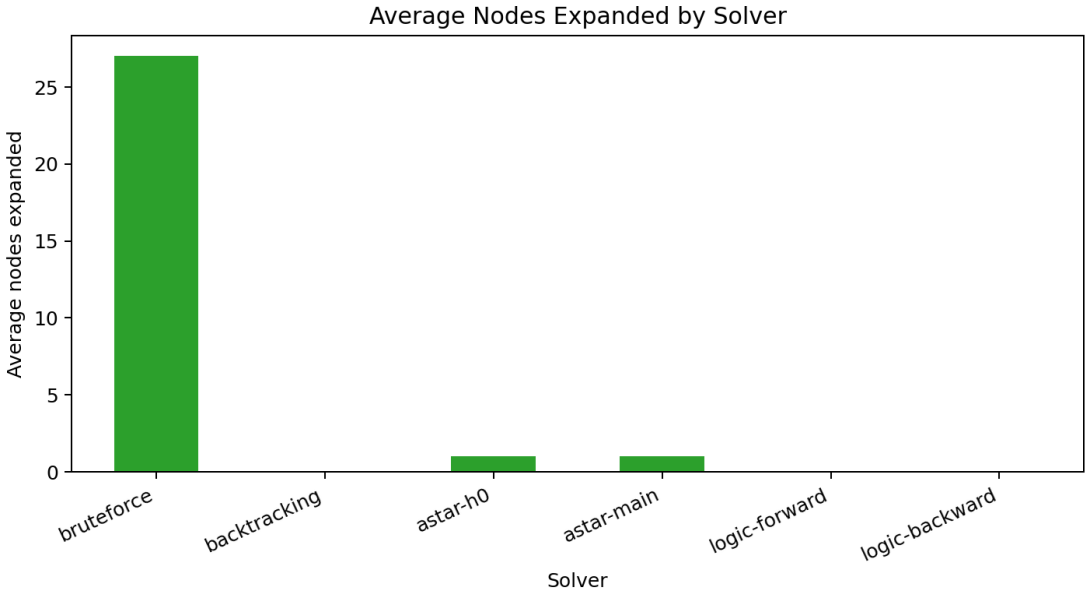

Figure 3: Average nodes expanded by solver.

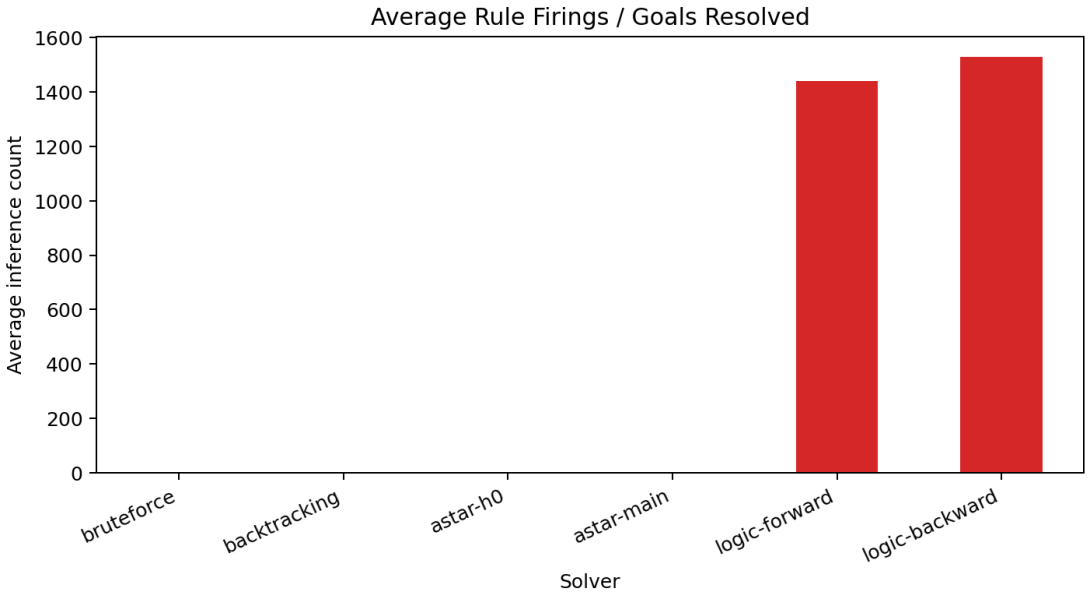

Figure 4: Average rule firings / goals resolved.

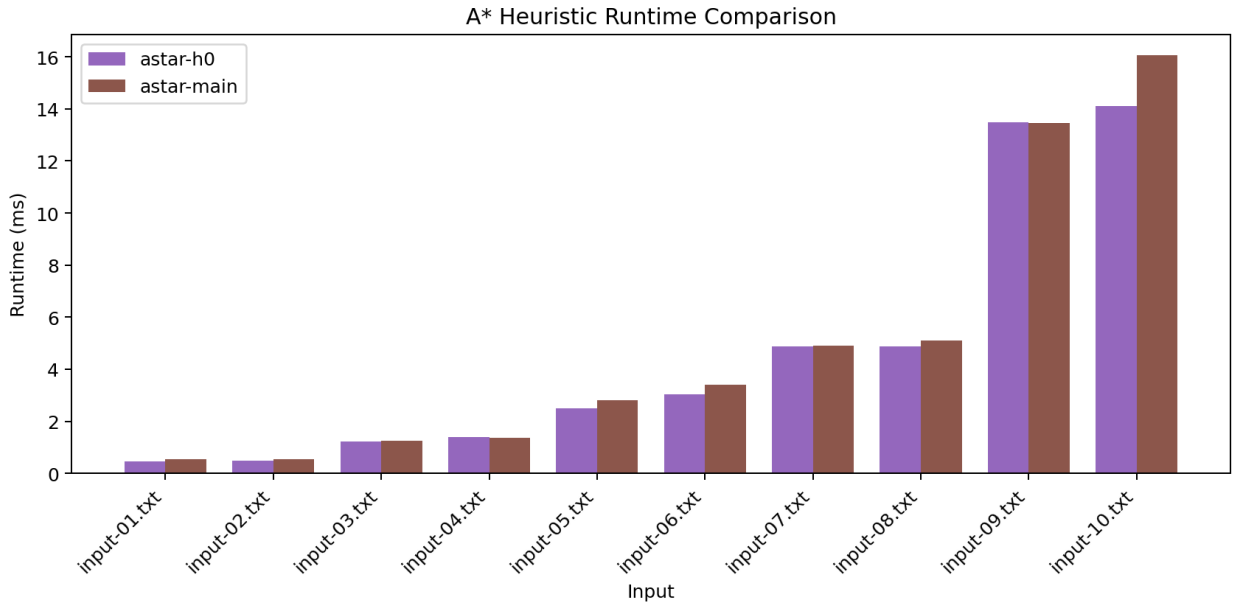

Figure 5: A* heuristic comparison on the ten required inputs.

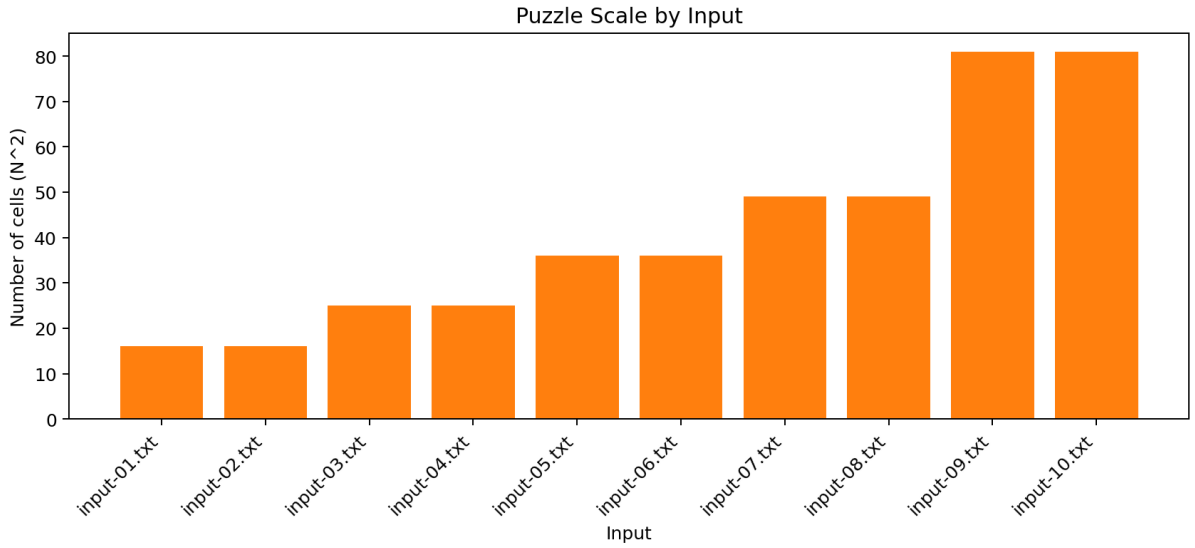

Figure 6: Puzzle scale by input size.

9 Comparative Analysis

9.1 What the Results Actually Show

The benchmark leads to four main observations.

1. The ten bundled inputs are propagation-friendly. Backtracking expands zero search nodes and A* expands only one node on every input. This means the dataset is strongly constrained: the combination of givens, row/column uniqueness, and inequality propagation is already enough to force each solution before deep search starts.

2. The A* heuristic is admissible, but its advantage is not visible on this dataset. The project requirement asks for an admissible heuristic, not a guaranteed speedup on every benchmark.

Here, h_{main} remains correct as a lower bound, but it is usually not better than h_0 in practice because AC-3 has already collapsed the search space. The runtime data is mixed: `astar-main` is slightly better on a few inputs and slightly worse on others.

3. The logic engines are now more academically honest than the earlier repository state. Forward chaining remains agenda-based Horn inference. Backward chaining now proves `Val(i,j,?)` through snapshot Horn rules instead of reading solved singleton values directly as `Val` facts. This makes `logic-backward` slower, but the behavior is closer to the project specification.

4. Brute-force is fast only because the benchmark set is easy for it. Although brute-force is the fastest solver in this benchmark, that result should not be over-interpreted. The inputs are structured and heavily constrained. On harder Futoshiki instances, brute-force would scale poorly.

9.2 Preferred Use of Each Method

- **Brute-force**: only as a naive baseline.
- **Backtracking**: best complete reference CSP solver in this repository.
- **A***: valid search formulation with a correct heuristic argument, but the current benchmark does not strongly separate it from propagation-heavy CSP search.
- **Forward chaining**: good demonstration of ground Horn inference; expensive because the rule base is large.
- **Backward chaining**: good demonstration of SLD-style querying; now academically cleaner but slower than forward chaining on large inputs.

10 GUI and Demonstration Support

The repository includes a Streamlit GUI with:

- bundled input selection and text editing;
- board visualization with horizontal and vertical inequality markers;
- solver execution and trace logs;
- benchmark view;
- backward-chaining query panel for `Val(i,j,?)`.

11 Self-evaluation Against the Rubric

Criterion	Status	Notes
FOL formalization and CNF derivation	Strong	Full axiom list included; three axioms derived step-by-step; Skolem-function dependency stated correctly.
Automatic KB generation	Strong	Ground Horn KB and direct CNF encoder both implemented for arbitrary N.
Forward chaining	Strong	Agenda-based inference from scratch; no hidden SAT/CSP solver.
Backward chaining	Stronger than before	Real SLD-style engine with unification; `Val` query now goes through rules in the snapshot program.
A* search and heuristic	Acceptable	Proper state, action, g, h, open set, and admissibility argument; empirical advantage is limited on the required dataset.
Comparison algorithms	Strong	Brute-force and CSP backtracking are both implemented and benchmarked.
Report and experiments	Stronger than before	Tables, charts, benchmark command, and Overleaf source now align with the code and the measured data.
GUI bonus	Present	Streamlit interface available for demo.

12 Demo Video

- `[Insert public URL]`

13 Limitations

1. The A* comparison remains academically valid but empirically weak because the ten required inputs are already easy for propagation. The report states this directly instead of exaggerating the heuristic.
2. Memory is reported as a proxy, not as operating-system RSS in MB.
3. The logic engines solve the current ten inputs without fallback, but that should not be generalized to every harder Futoshiki instance.
4. The direct CNF encoder is used for verification, not as the primary solver, in order to keep the forward-chaining, backward-chaining, and search requirements honest.
5. The task-distribution section still needs the real team names, IDs, and percentages from the actual group.

14 References

References

[1] CSC14003, *Project 2. Logic – Futoshiki Puzzles.*

[2] CSC14003 Lecture 02 Part 1, *Problem Solving by Search – Search Problem Formulation.*

[3] CSC14003 Lecture 02 Part 2, *Problem Solving by Search – Global Search Strategies.*

[4] CSC14003 Lecture 02 Part 4, *Problem Solving by Search – Heuristic Functions.*

[5] CSC14003 Lecture 04, *Constraint Satisfaction Problems.*

[6] CSC14003 Lecture 05 Part 1, *Knowledge and Reasoning – Propositional Inference.*

[7] CSC14003 Lecture 05 Part 2, *Knowledge and Reasoning – First-order Inference.*